BUILD AN AGENT CREW



F.A.M-E.D. | Assignment 3 Submission

Assignment 3 | Piero L. S. Folly | Khi-Rho Studios | 1 November 2026

F.A.M-E.D. (Fire Age Main-Eterra Dungeoneers) is a cooperative exploration and deck-building game. The submission contains the crew roles, orchestration evidence, card data, checks and an existing Bauglor combat prototype.

The report connects nine prototype results and my play feedback to a proposed production workflow. The assignment deliverables are mapped below.

Required deliverable	In this submission
Crew Code	Role prompts, deterministic gates, a recorded-output runner and the six-card offline controller. The model-dispatch records are available on request.
Mermaid Diagram	The editable crew-diagram.mmd, also drawn in the README. My sketch, the Group 10 graph and the proposed flow are in this report.
README	Named game, output, crew roles, folder map and evidence limits.
Game-ready output	An existing Group 10 combat build and its card and rule data, shown on request. It is a tested prototype, not an approved Unreal import or a final game.

What ran and what is a replay

I kept the records of four separate Group 10 model calls. They are not in this repository; I can share them on request. The Lead chat used its native Agent tool to run those workers. The supplied replay makes no new model calls. The older six-card A3 controller has no live transport, and its release gates remain open.

2. THE GAME AND THE OUTPUT

Four character roles face enemies on a combat grid. Cards provide actions and resources. Combat is the first production target; dungeon encounters and landscape exploration follow.

Loop	What the crew must support	Current boundary
[Combat]	Card piles, movement, attack, defence, meaningful card choices, bots, victory, defeat and restart.	The nine versions explore one Bauglor fight. They use disclosed prototype values and do not prove complete rulebook fidelity.
[Dungeon Crawl]	Encounters, persistent tiles and the transition into combat.	Later. The demos mostly start directly in the fight rather than proving the full encounter-reveal loop.
[Landscape]	Exploration, events and the route between places.	Later. Not built or verified by this submission.

A small, concrete production task

The prepared card crew targets six stable row IDs. A stable ID lets the game, data and tests refer to the same card without guessing from its display name. It preserves raw source text separately from parsed fields.

ID	Card	Intended use
SK1_01	Knockout!	Source-checked card row; real print comparison and approval still required.
SK1_02	Double Strike	Source-checked card row; real print comparison and approval still required.
SK1_03	Mental Push	Source-checked card row; real print comparison and approval still required.
SK1_04	Salvage Resources	Source-checked card row; real print comparison and approval still required.
SK3_01	Stand as One	Source-checked card row; real print comparison and approval still required.

ID	Card	Intended use
SK4_01	Edan's Leap	Source-checked card row; real print comparison and approval still required.

What is usable now

Group 10 supplies an existing web combat build, card data, a rule audit, checks and a crew trace. The separate six-card code supplies reproducible offline diagnostics. It holds back DT_Cards.csv when the printed card cannot be confirmed. That stop is useful: the crew must not turn uncertain card text into game data.

The future consumer is the F.A.M-E.D. Unreal card table. I have not tested an Unreal import yet.

3. THE STUDIO THAT DID THE WORK

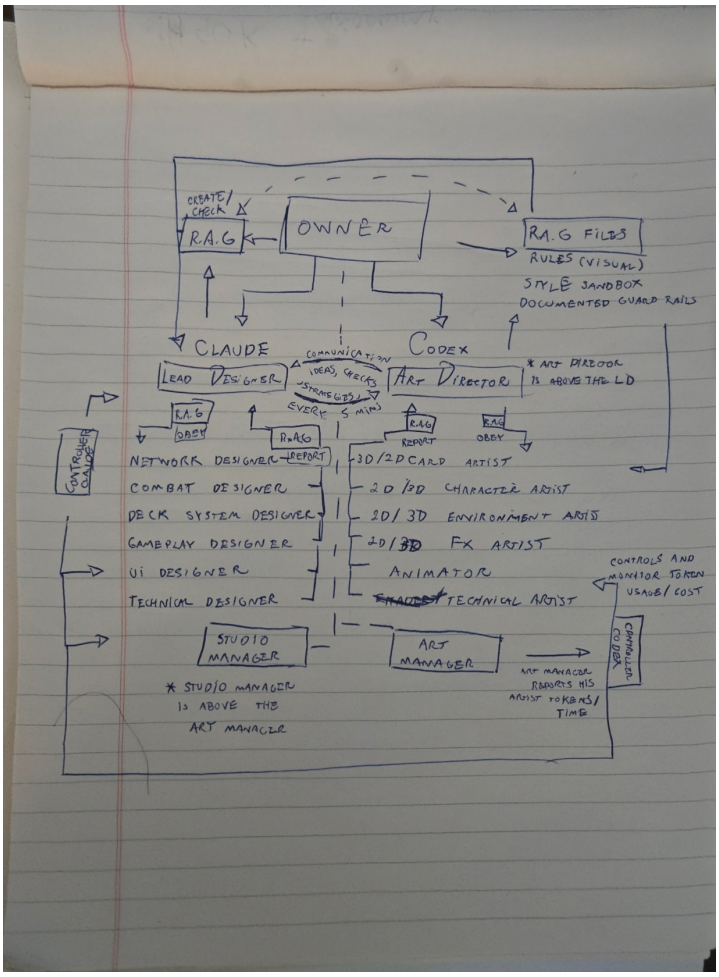
My original sketch separated design and art leadership, approved source material, specialists and cost control. The implemented studio added named inboxes, dated evidence and small scripts. The source drawing and the task pipeline are shown separately below.

Role	Actual responsibility	Boundary
Piero Folly, founder	Sets the game, visual direction, priorities and final choices; plays and comments on the prototypes.	My comments stay distinct from agent interpretation. Only I choose clearing and deletion.
Studio Director, Codex	Coordinates the studio and answers the questions I delegate while I am away.	Delegated answers are labelled. It does not turn an unknown fact into my decision.
Art Director, Codex	Directs presentation, sources and independent checks; coordinated the two Codex builds and this submission.	A source review is not a play test. It keeps the original evidence and says what was checked.
Studio Manager, Claude	Tracks work, model settings, usage, inbox coverage, unresolved items and handoffs.	Alerts and reports do not enforce security or spending limits.
Lead Designer, Claude	Runs the Claude production stages, integrates results, maintains the shared pages and coordinates the single push.	Owns the shared studio pages.
Lead Engineer, Claude	Prepares the local environment and engine support; records technical limits.	Installed tools do not mean a working Unreal game yet.
Lead Web Designer, Claude	Maintains navigation, publication surfaces and the readable artifact experience.	Presentation changes keep the existing page and its history.

A task needs more than a role name

Each task should name its game and folder, approved inputs, output format, test, budget and stop condition. The producer owns its files. A checker reports a fault before the producer corrects it. The manager tracks the handoff, and I settle a genuine design choice.

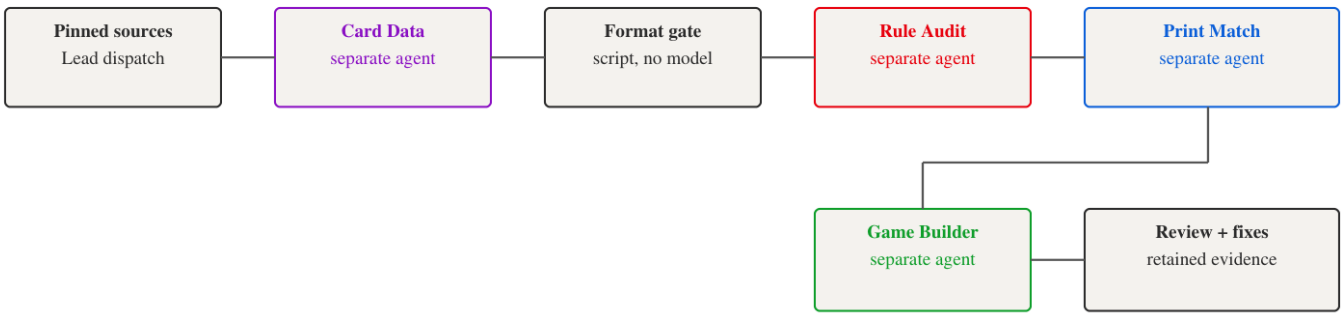
The studio used real model calls, local scripts and my decisions. It is not a fully autonomous company.



My original design
My pen-and-paper sketch from 29 October. The drawing places me above Claude's Lead Designer and Codex's Art Director, with scoped source material, specialist roles and cost monitors.
Evolution into task crews
The sketch describes studio oversight. The production graphs describe a smaller task: sources become checked card data, then implementation. Controllers and dated handoffs support that flow. The RAG and sandbox in the drawing were not built.

4. ONE WORKING CREW: FORMAT FIRST

Group 10 is the clearest small case for this submission. The Lead called separate workers for Card Data, Rule Audit, Print Match and Game Builder. I kept their call IDs, models, inputs and times; they are available on request, not in this repository.



Working case: Group 10, native Agent calls. This is not the held six-card replay.

Initial print inputs and later corrections are preserved; no ideal blind-process claim.

Step	Input to output	Why it exists
Card Data	Approved packet -> combat-data.json	Make structured rows before writing the game.
Format gate	Rows -> pass or typed findings	A cheap script catches a wrong shape without another model call.
Rule Audit	Rows and sources -> audit.json	A second worker checks meanings and unresolved decisions.

Step	Input to output	Why it exists
Print Match	Card images and packet -> print evidence	Compare the rendered card with the data. The first Group 10 pass used art-only images; the later correction is shown.
Game Builder	Accepted data, audit, print notes and rulings -> playable build	Connect the material to an actual game loop.
Review and correction	Produced files -> defects, fixes and checks	A successful first build is not the end of validation.

Group 10 needed corrections, including last-card falling, a buff expiry case and defence use. Its initial print step did not establish the ideal blind source comparison. I kept those faults in the record.

5. THE CODE AND THE NEXT CARD CREW

The actual game crews used the platform Agent tool as their model transport. I kept those task inputs; they are available on request. Their gates and game code are ordinary source files.

The separate six-card controller is ordinary Python. It prepares a pinned packet, validates each response and keeps the first blind print read. The live transport is deliberately absent. With the private inputs, this code replays saved responses. This public copy includes only the code and a saved replay record; it cannot rerun the full pipeline.

Mechanism	Implemented behaviour	Limit
Pinned sources	Checks the allowed inventory and file fingerprints before stages and attempts.	A fingerprint proves unchanged bytes, not that the source rule is correct.
Three distinct tasks	Card Data produces; Rule Audit checks; Print Match reads only names, IDs and pinned images in the proposed blind flow.	The older offline code alone does not prove live calls. The separately recorded fresh native run is in section 13.
Response shape	Each role returns a defined JSON structure. Wrong format retries only that stage.	Four attempts per stage; the global ten-call cap wins if limits conflict.
Global caps	MAX_CALLS = 10; MAX_TOKENS = 400000 for recorded usage in replay.	No claim that this controls every studio chat or the provider account.
Stop routes	Source mismatch, usage-limit stop or exhausted retry leaves diagnostics.	No paid API, login, account switch or automatic reset fallback.
Print and release	If the printed card cannot be confirmed, the crew keeps its QA notes and does not release the CSV.	Human review and real source/print approval remain required.

Plain orchestration recipe

```
pin_and_check_sources()
rows = call_role("Card Data", source_packet)
check_format(rows)
audit = call_role("Rule Audit", source_packet, rows)
print_read = call_role("Print Match", images_ids_names_only)
keep_first_read_unchanged(print_read)
result = validate(rows, audit, print_read)
export_only_if_eligible(result)
record_inputs_outputs_usage_and_stop_reason()
```

The recipe describes control flow. Section 1 says which parts ran live and which are a replay.

6. OUTPUTS, CHECKS AND FAILURE

Done means an output and evidence for that output. A model saying “pass” is one claim. A script checking the file, another worker finding a defect, and me playing the game are separate observations.

Evidence	What it supports	What it does not support
Four original native calls	Separate Group 10 role dispatches with recorded task inputs and models.	Perfect isolation, every chart box, or all rulebook rules.
Produced game and data files	The named workers produced material connected to a playable F.A.M-E.D. prototype.	Final game balance or production readiness.
Group 10 focused corrections	Fault reports and later checks cover specific falling, expiry and defence cases.	A complete mathematical proof of the game.
My feedback	I played the nine versions and wrote down what I liked, what confused me and what broke.	A controlled usability study or a human loss/restart test for every game.
Six-card historical tests	The correction run recorded 34 passes, zero failures and zero new model calls. A later combined run recorded 65 passes and one skip.	A fresh test run today or a live three-model crew.
Replay result	The controller code and the replay log are in the repository. The source assets are withheld.	Real print legibility, a fresh model run, or an Unreal import.

What the card pack must keep

The row key and display name stay separate. Raw card wording stays beside parsed values. Timing, after-use pile, tags, quantity and source references are checked. Unresolved template bindings stay unresolved. Printed exceptions are not replaced by a model's preferred rule.

A safe stop is an output

If a card is unreadable, a higher-ranked source is missing or an approval is open, the crew records the gap and stops release. It should keep the attempted response and the reason. It should not silently create another account, call a paid service or rewrite a previous result.

7. NINE PROTOTYPES, DIFFERENT VARIABLES

Nine versions explored the same general Bauglor fight: two Codex builds and seven Claude builds. The planned four-plus-four comparison was not completed. Group 09 and the final ninth are different builds.

Game	Builder	Engine	Nominal crew method	Main observation
02	Claude	Web	Sequential baseline	Readable log and tabs; autoplay complaint.
03	Codex	Canvas web	Adapted Chart03	Useful trace; weak visual guidance.
04	Claude	Godot	Joined data and code	Fewer redundant actions; space-bar complaint.
05	Claude	Phaser web	Separate integration lane	Strong styling; too many prompts.
06	Codex	Canvas web	Adapted Chart06	Least liked; avoid the direction.
07	Claude	HTML/CSS/SVG web	Contract and oracle	Smooth board/card flow; adjacency confusion.
09	Claude	Three.js web	Evidence chain	Minimal 3D and camera control.
10	Claude	Phaser web	Format first	Animation and concise feedback.
ninth	Claude	Web	Sealed-oracle chain with lanes	Helpful controls; divided attention.

What changed between crews

Engine, worker count, task separation, checks, retry routes, art and interaction all varied. Some builders performed several nominal chart roles. Some independent stages ran later. The source files distinguish the planned graph from the actual trace.

Comparison limits

The set reveals interface and workflow ideas. Engines, task boundaries, checks and presentation varied together, so these results cannot establish a causal model or crew winner.

The next comparison should use the selected features and the same named acceptance cases. That future controlled test has not been run.

8. MY FEEDBACK AND THE DIRECTOR'S READING

I gave written feedback, not numerical grades. The scores below are Studio Director interpretations. They are not averaged into a winner. Exact original message times are unknown.

Game	Director interpretation	My words, excerpt	Keep or address
02	8/10	the Combat log font format + colors make the game very easy to follow.	Readable log and tabs; autoplay complaint.
03	4/10	This is the weakest game that I played so far.	Useful trace; weak visual guidance.
04	8.5/10 provisional	My two most favorite so far were the Godot, the Godot one, and right now this one, the Group 10	Fewer redundant actions; space-bar complaint.
05	7.5/10	the strongest thing here is the aesthetics and the amount of icons and explanation, but that can be overwhelming too.	Strong styling; too many prompts.
06	3/10	I think this one's the one that I like the least, the number six, group six.	Least liked; avoid the direction.
07	9/10	This one has actually been the highest grade so far.	Smooth board/card flow; adjacency confusion.
09	7/10	I feel like the most outstanding thing is that you can rotate the camera on this other one in the localhost	Minimal 3D and camera control.
10	9/10	But this game has the best aesthetics in terms of interaction, animation, how being concise and in a unique language.	Animation and concise feedback.
ninth	6/10	It definitely looks like a game made for an engineer, not for a gamer.	Helpful controls; divided attention.

Preferred three prototypes

My preferred three were 04 Godot, 07 smooth 2.5D/isometric and 10 pixel art. Preferred features were fewer redundant actions in 04, board/card flow in 07, and animation with concise guidance in 10. Group 02 adds the preferred log formatting.

Problems stay beside praise

Faults I reported include autoplay not turning off in 02, a space-bar reset problem in 04, missing log scroll in 10, unclear diagonal adjacency in 07, and divided attention in the final ninth. These faults were not newly reproduced for this document.

The Art Director checked these rows against the source.

9. SELECTED DESIGN AND CREW

The next design should keep the board and current action visually dominant. Extra information should explain the current choice without competing for attention.

Borrow	Use it for	Acceptance question
07: board focus and card hover	Keep the player's eye near the action and explain a card before committing it.	Can a new player identify the active hero, legal target and card result?
10: animation and event guidance	Show what changed through concise motion, dice feedback and central prompts.	Does feedback explain the action without hiding the next choice?
04: fewer clicks	Automate a second step only when no meaningful intervening action is available.	Does automation stop when a player can legally respond?
02: readable log	Use tabs, clear type, colour and a scrollable history.	Can the player find who acted, what happened and why?
09: camera and minimal UI	Keep optional camera control and show only current information.	Can the player return to a useful view without losing state?
03, 05 and ninth: inspection tools	Keep trace, rules, seed, card details and accessibility behind clear controls.	Can the player inspect detail without moving the board out of focus?

Crew choice

The proposed crew combines source extraction, an early format gate, independent rule and print checks, a checked contract with expected cases, implementation, and independent review. Rules and presentation remain separate.

Combat before expansion

First, make one readable **[Combat]** loop with the right card decisions and reliable endings. Next, connect **[Dungeon Crawl]** encounters and persistent tiles. Only then grow the **[Landscape]**. Shared IDs and approved event contracts can carry the data forward; each phase still needs its own sources and tests.

This is a proposed direction. I have not built a tenth game or picked a final crew.

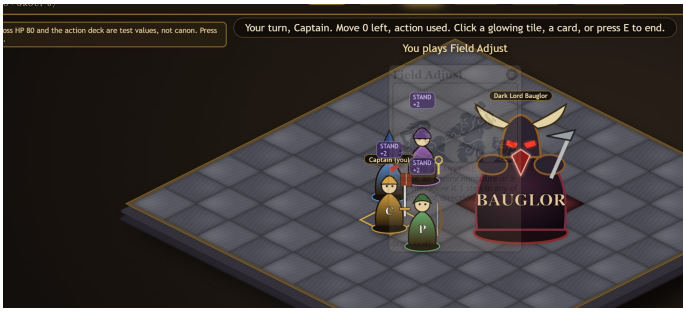
Why these three, and why this graph?

Preferred prototype	Workflow records kept	Reason for the combination
04 - Godot; Joined data and code	Checked rows and expected cases feed the builder.	Connect rule data to implementation; retain the fewer-clicks direction.
07 - smooth isometric; Contract and oracle	An allowed-value contract and hand-computed cases constrain implementation.	Make accepted rules and expected results explicit; keep the clear board and card hover.
10 - pixel art; Format first	A deterministic format gate precedes later rule, print and build work. Independent review found defects after initial acceptance.	Reject structural errors early, then require a separate critic and recheck; retain animation and focused event guidance.

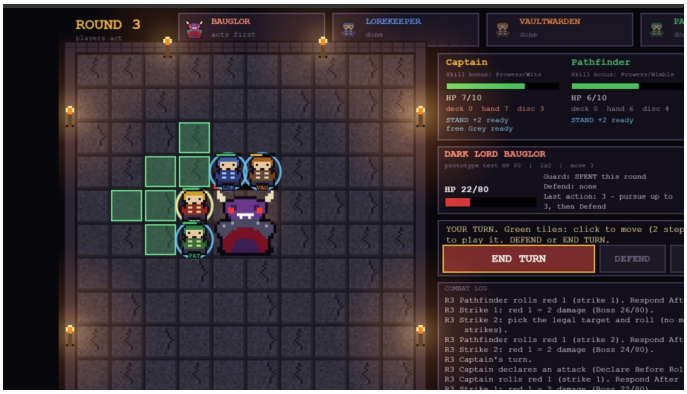
My preference selects the interface direction. Traces select useful process mechanisms. The recommended combination is a proposal; it does not prove that one chart caused a better game. The Group 10 graph in section 4 shows the clearest four-worker case: a crew that really ran. It did not win every comparison.

Play-test snippets

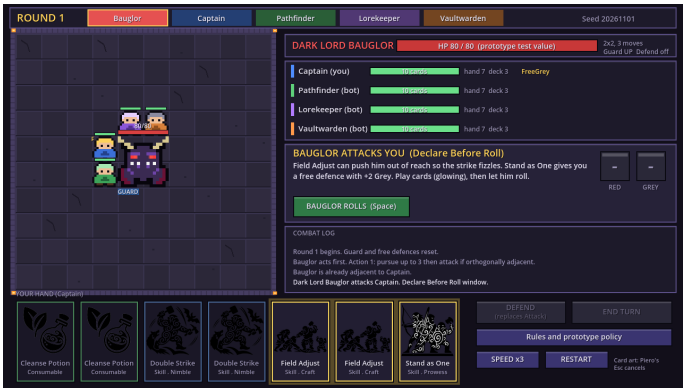
Small crops from my play test, with the Windows watermark and browser bars removed. Each shows the feature that led to keeping that crew's mechanism.



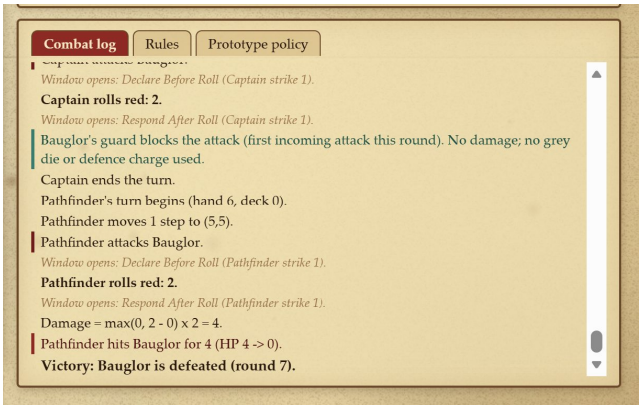
07, Contract and oracle: smooth 2.5D board and card play. The contract and expected cases made the rules explicit before implementation.



10, Format first: pixel art, guided turn and readable panels. The format gate rejected structural faults before later stages; independent review still found defects.



04, Joined data and code (Godot): one combat agent owned the turn flow. Builder screenshot; I took none of this one.



02, Sequential baseline: the tabbed, colour-coded combat log I want reused.

Proposed combined flow

Pinned sources -> Card Data -> format gate -> independent Rule Audit and blind Print Match -> checked contract and expected cases -> builder -> independent rules/interaction review -> one correction and recheck. Unresolved rule or print evidence stops release. Visual changes must not alter combat rules.

10. THE ARTIFACT, CUP AND CLEAR

The studio pages show the work. The repository and dated records keep the evidence. A useful page says what changed, when it was captured, who did it, what was checked and what remains open.

Surface	Purpose	Limit
Studio Home	Navigation, current work, decisions and links.	A page label can go stale; the source file is the record.
Crew Tests	The nine games, diagrams, reports, my feedback and comparisons.	Planned roles, actual roles, test volume and my preference must remain separate.
Game Dev Logs	Per-game changes, bugs, authors and evidence.	Later entries keep their real logging date.
General Dev Log	Cross-studio events and task summaries with dates and times.	A local log line, a saved record and a visible row can differ.
Mailboxes	A specific role receives a task, question, blocker or result.	A message in an inbox is not yet handled. Handled messages are not repeated.

CUP: Commit, Update, Push

CUP means Commit, Update, Push: inspect and commit attributable changes, update the relevant log and handoff, then push from one place. Private runtime and unrelated files stay out.

Clear is a handoff, not deletion

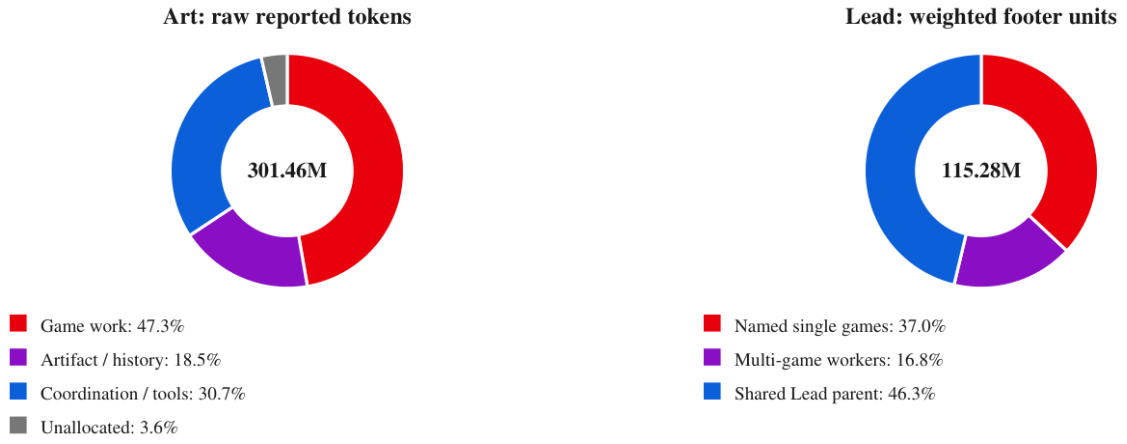
A clear requires a handoff of task state, open questions, source paths, costs, mistakes and running processes. I choose the clear.

The studio caught stale counts, incorrect authorship labels, unreliable times and missed inbox messages. Corrections were added without erasing the earlier record. I learned to say what was checked, not just “synced”.

11. KEEPING THE WORK UNDER BUDGET

The manager tracks usage and routes work. Deterministic checks do the work that does not need a model. Short role packets, few retries and saved responses reduce unnecessary repeat calls. A budget alert must lead to an action.

Control	What it does	Limit
Card controller caps	Stops its recorded-run loop at ten calls or 400000 recorded tokens; retries only format failures.	It is not a studio-wide spending limit.
Codex budget reader	Reads quota metadata and produces deduplicated warnings, including an unavailable-source warning.	It does not stop model work or set a spending limit.
Claude budget guard	Uses saved meter readings and estimates to emit alerts.	An old threshold in code may not match my latest rule; someone still has to stop the work.
Studio Manager controller	Checks stale work, inbox coverage, watcher state, budgets and rule drift.	It is not a security sandbox or a full health check.
Recorded task splits	Keep main coordination separate from direct game workers.	It is not a full price per game or a quality-per-token score.
Reserve and hold	Preserve at least 10% of Codex weekly allowance; no automatic reset, account switch or paid fallback.	An account-wide meter change is not this task’s cost.



Separate snapshots and units. Neither chart measures active time or dollars.

Chart: **recorded workload, not elapsed time**. The left groups Art’s task-window records into game work, artifact/history work, coordination/infrastructure and unallocated work. It includes repeated cached input and is a correlation, not a bill. The right uses Lead’s weighted footer units from its separate snapshot. Each chart has its own total and scale. They are never added; shared work is not split equally between games.

No reliable active-time split or per-task dollar bill is available. The charts show measured workload units only.

12. THE JOURNEY AND WHAT CHANGED

Period	Work and change	Lesson
26-28 October	Source rules, GDD revisions, six-card scope, response schemas and source boundaries were prepared.	A stable row ID and a pinned source packet are better than a large unfiltered folder.
28-29 October	The offline controller, retry routes, print boundary and failure cases were corrected.	A test with made-up data is not a real print check.
31 October	Document/code review, activity capture, handoffs and watchdog work exposed coordination gaps.	Each role needs its own inbox and a clear result.
1 November, overnight	Crews produced nine combat variants with different charts, engines and interfaces.	A plan on paper differs from what the crew actually ran. Crews of unequal size cannot pick a winning model.
1 November, my play test	My reports shifted the emphasis toward board focus, readable feedback and fewer clicks.	A game that passes scripts can still be confusing or unpleasant to play.
1 November, close-out	Before each clear, I saved the logs, counts, commits, token splits and mistakes.	Keep the reason for a correction, not only the final result.

Mistakes that improved the system

A sender-prefix filter missed legitimate messages. A blurred screenshot produced an unreliable log count. Some later records named the wrong author. A source-only check was at risk of being reported as live confirmation. Each mistake led to a clearer label or a better check.

The merged mistakes register has 77 recorded rows across six chats at its saved cutoff. Rows include incidents, blocks and game defects. They can overlap across chats, and environment blocks are not model mistakes. The total is **not an AI error rate**.

Time spent

The record spans several calendar days, not continuous labour. Group 10's saved stage durations add to about 33 minutes, while its event span is 7 hours 20 minutes 43 seconds including idle gaps.

13. A FRESH THREE-AGENT CARD RUN

The fresh run I authorized used three separate Codex workers (records available on request): Card Data, Rule Audit and blind Print Match. Each returned six structured rows. There was one call per role, no retry, no paid API and no new game.

Role	Actual input and result	Note
Card Data, Sol Low	Read one pinned source packet; returned six exact-text card rows plus fields and unresolved items.	A correct format did not mean valid game data. The check script rejected wrong source labels and unsupported effect tags.
Rule Audit, Sol High	Read independent pinned sources and the fresh producer rows. Flagged four source reviews; passed two rows.	Flags are review findings, not four automatically confirmed bugs. The missing structured value on Knockout needs interpretation rather than an invented ruling.
Print Match, Sol High	Viewed only six exact pinned image crops, IDs and names. Read five; marked Mental Push unreadable.	It saw no producer or audit text. It checked screen images, not the physical printed cards.

The result: stop before release

Stopped for my review; no CSV was released. The existing deterministic validator checked all three outputs without a crash. It kept the first blind response unchanged. Source review flags, wrong producer tags and source labels and unreadable print blocked release. No source was changed to make a green result. Group 10 remains the playable game-output case: a private prototype, available on request.

What the critic caught

Double Strike says +4 in exploration text but its structured value says 3. Salvage Resources names the R.F.D. pile in text while a structured field names discard. Mental Push has unresolved reward symbols and a round/turn disagreement. These are specific, cited findings for later source resolution. The crew kept raw text rather than guessing a fix.

Actual controls and limits

The workers were told to read only one source packet or the six images. This adapts the older plan; it is not a hardened sandbox. Per-worker token counts were not available.